

Search with Zero or Partial Percepts

The agent **may not know the current state** in a partially observable environment.

Outline

- I. Searching with no observation
- II. Searching in partially observable environments
- III. Online search agents and unknown environments

I. Sensorless Situation

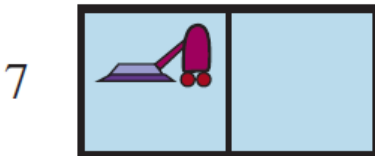
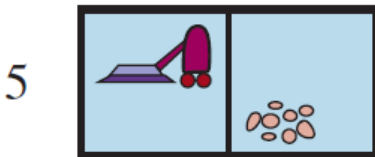
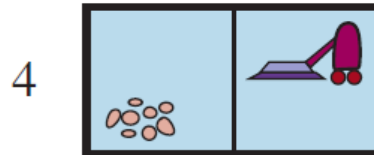
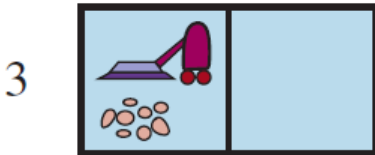
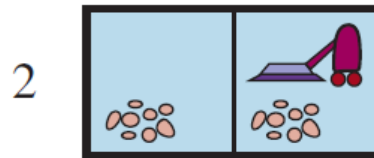
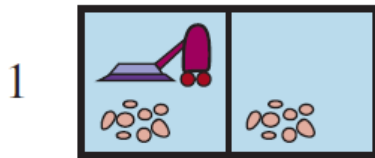
Agent's precepts provide no information at all.

- ♦ Sensorless solutions are surprisingly common.
- ♦ They can be *robust* due to zero dependence on sensors working properly.

I. Sensorless Situation

Agent's precepts provide no information at all.

- ◆ Sensorless solutions are surprisingly common.
- ◆ They can be *robust* due to zero dependence on sensors working properly.

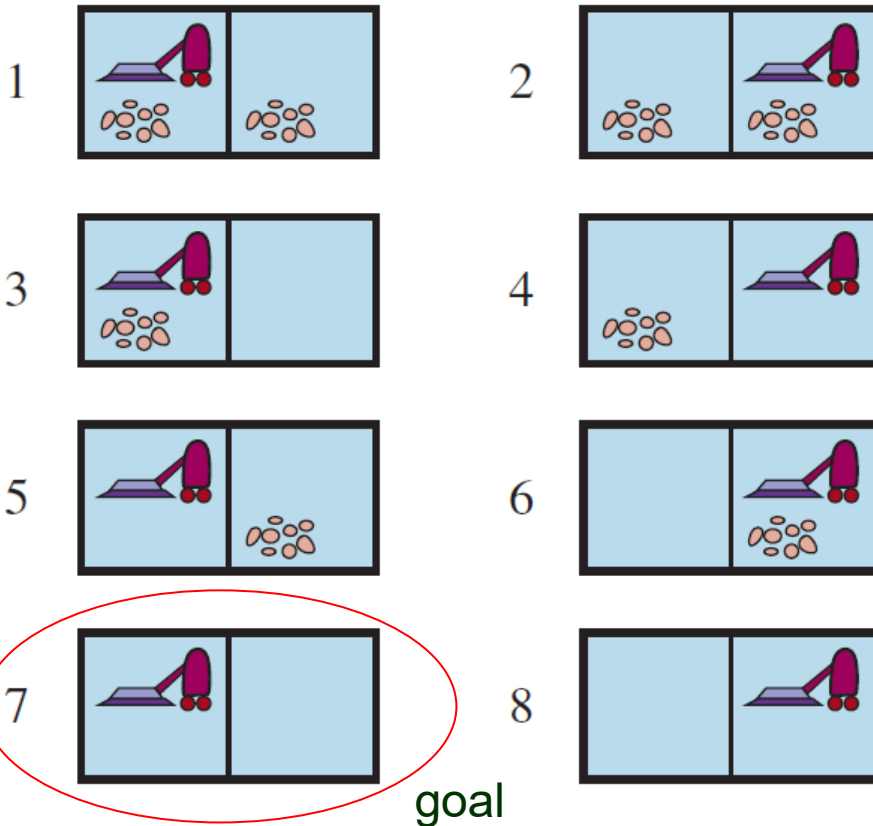


- Knowledge of geography
- No knowledge about location and dirt distribution

I. Sensorless Situation

Agent's precepts provide no information at all.

- ♦ Sensorless solutions are surprisingly common.
- ♦ They can be *robust* due to zero dependence on sensors working properly.



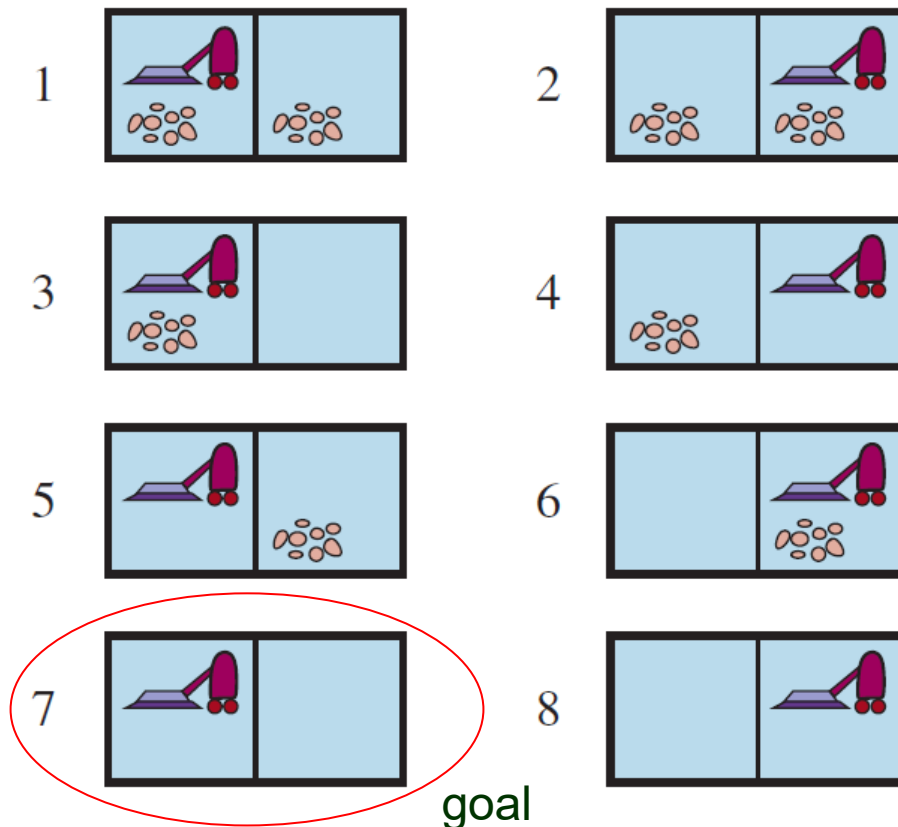
- Knowledge of geography
- No knowledge about location and dirt distribution

{1, 2, 3, 4, 5, 6, 7, 8}

I. Sensorless Situation

Agent's precepts provide no information at all.

- ♦ Sensorless solutions are surprisingly common.
- ♦ They can be *robust* due to zero dependence on sensors working properly.



- Knowledge of geography
- No knowledge about location and dirt distribution

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

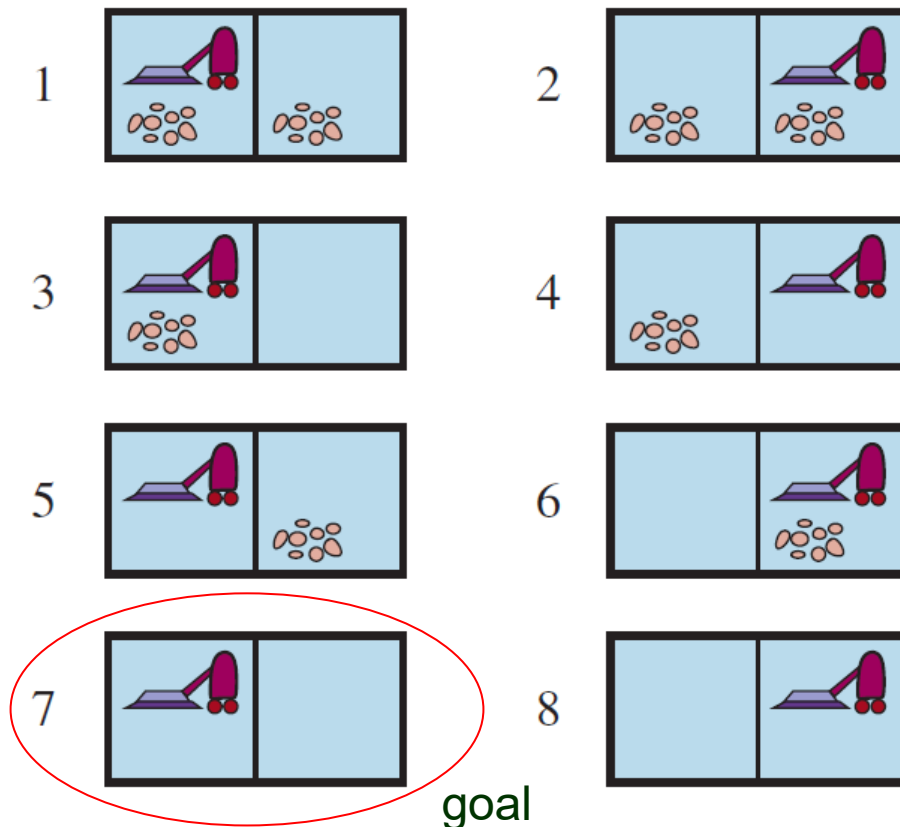
↓ *Right*

$\{2, 4, 6, 8\}$

I. Sensorless Situation

Agent's precepts provide no information at all.

- ♦ Sensorless solutions are surprisingly common.
- ♦ They can be *robust* due to zero dependence on sensors working properly.



- Knowledge of geography
- No knowledge about location and dirt distribution

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

↓ *Right*

$\{2, 4, 6, 8\}$

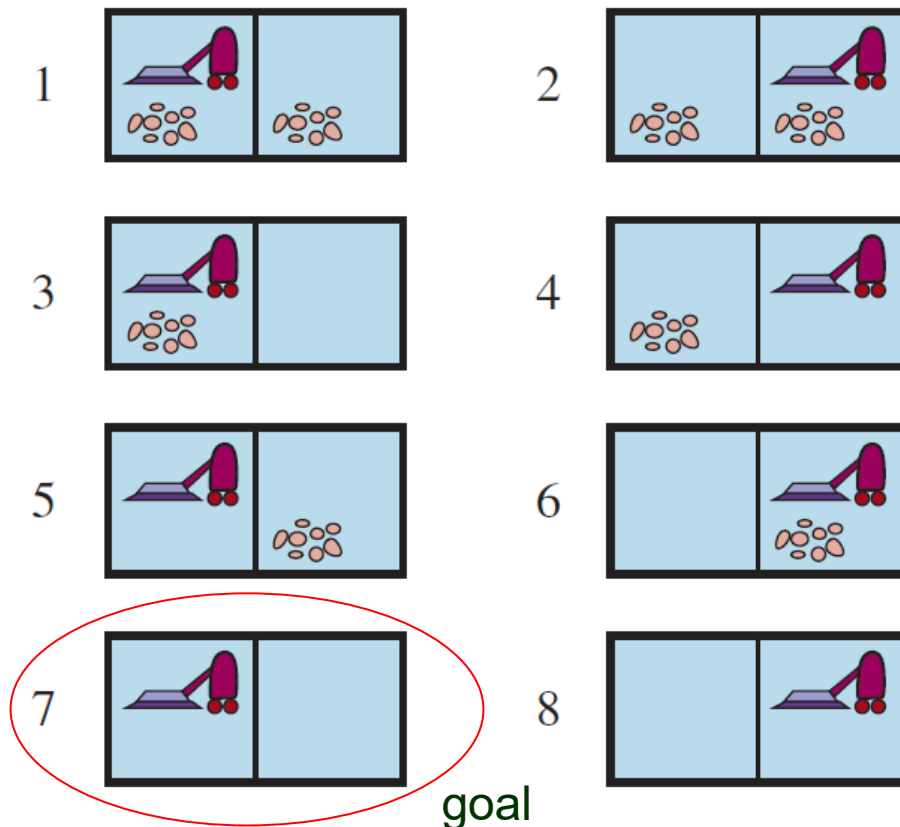
↓ *Suck*

$\{4, 8\}$

I. Sensorless Situation

Agent's precepts provide no information at all.

- ♦ Sensorless solutions are surprisingly common.
- ♦ They can be *robust* due to zero dependence on sensors working properly.



- Knowledge of geography
- No knowledge about location and dirt distribution

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

↓ *Right*

$\{2, 4, 6, 8\}$

↓ *Suck*

$\{4, 8\}$

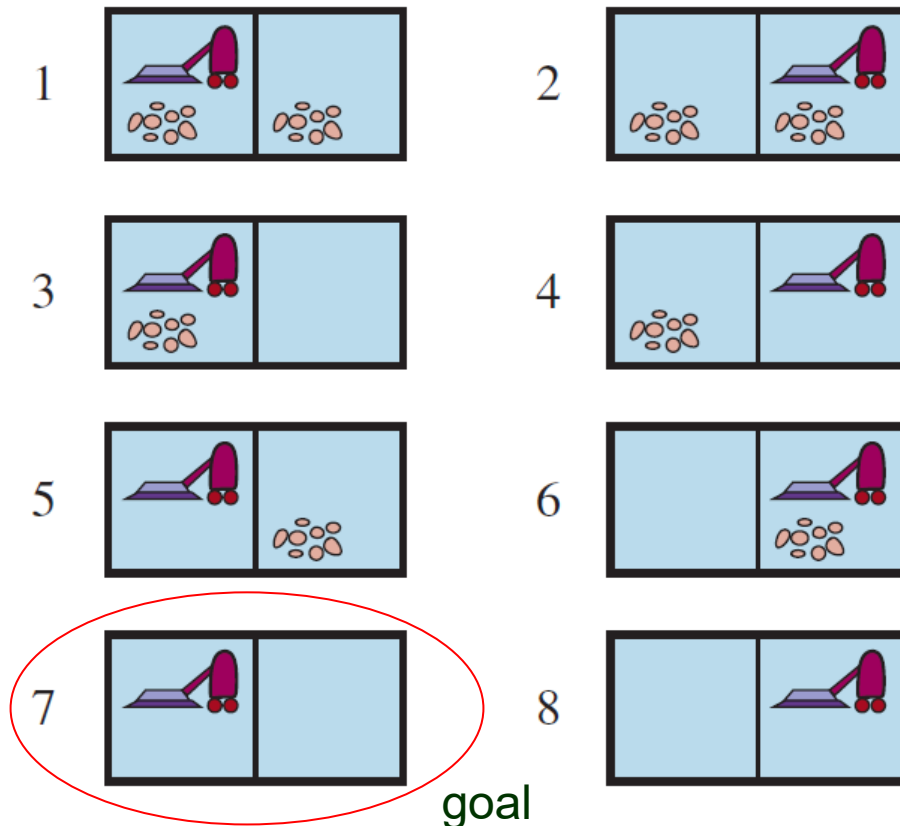
↓ *Left*

$\{3, 7\}$

I. Sensorless Situation

Agent's precepts provide no information at all.

- ♦ Sensorless solutions are surprisingly common.
- ♦ They can be *robust* due to zero dependence on sensors working properly.



- Knowledge of geography
- No knowledge about location and dirt distribution

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

↓ *Right*

$\{2, 4, 6, 8\}$

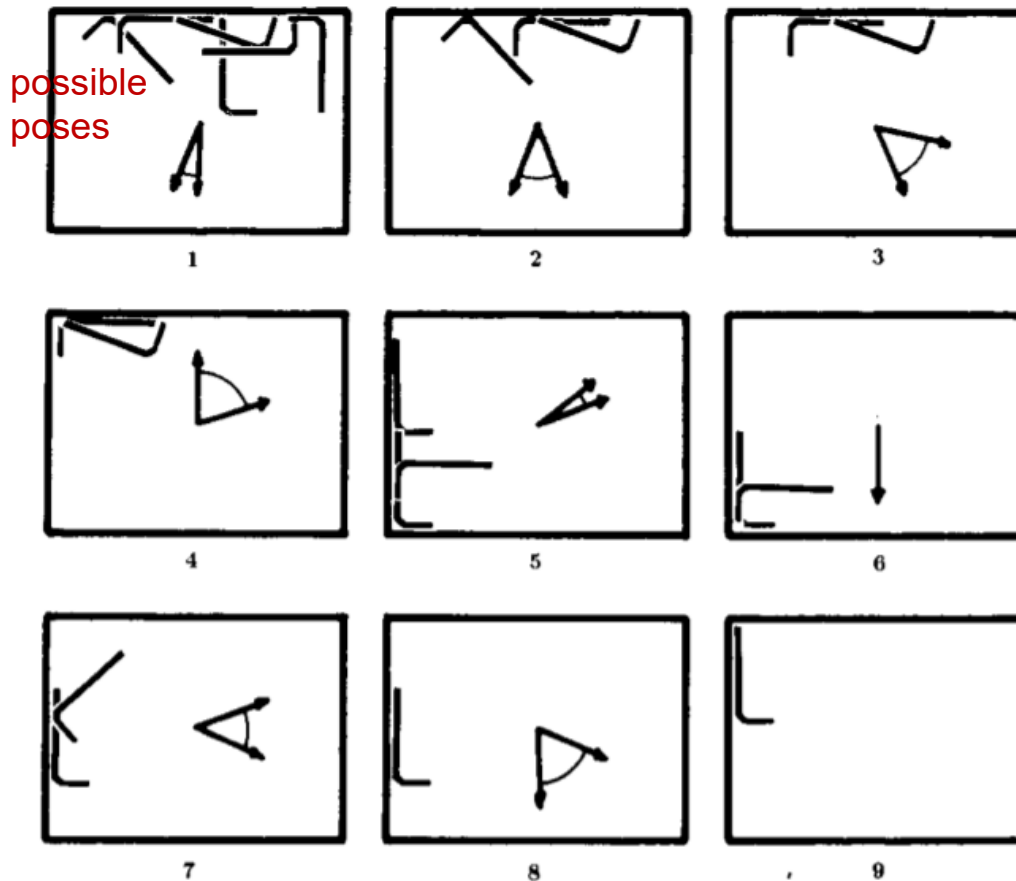
↓ *Suck*

$\{4, 8\}$

↓ *Left*

$\{3, 7\} \xrightarrow{\text{Suck}} \{7\}$

Sensorless Manipulation – Tray Tilting



Erdmann & Mason (1988):

Orients the Allen wrench
with eight tilts.

- Slow planar motion
- Coulomb friction

[IEEE Journal of Robotics and Automation,
vol. 4, no. 4, pp. 369-379, 1988.](#)

Fig. 2. Beginning at the upper left and moving from left to right, we can trace an automatically generated program that orients the wrench. Each frame shows the set of possible wrench contacts, and the operation to be applied. Each operation is represented by an interval of azimuths. The azimuth arrows indicate the tray's direction of steepest ascent; gravity acts in the opposite direction.

Solution to a Sensorless Problem

A sequence of actions from search in the space B of *belief states* (*b-states*).

- ♦ Full observability in B : the agent always knows its belief state.

Solution to a Sensorless Problem

A sequence of actions from search in the space B of *belief states* (*b-states*).

♦ **Full observability** in B : the agent always knows its belief state.

1. Transform the physical problem into a belief state problem.
2. Apply existing search algorithms.

Solution to a Sensorless Problem

A sequence of actions from search in the space B of *belief states (b-states)*.

- ♦ **Full observability** in B : the agent always knows its belief state.

1. Transform the physical problem into a belief state problem.

2. Apply existing search algorithms.

- **States (b-states)**: 2^N possible subsets of the set Σ of N physical states.

Solution to a Sensorless Problem

A sequence of actions from search in the space B of *belief states (b-states)*.

♦ **Full observability** in B : the agent always knows its belief state.

1. Transform the physical problem into a belief state problem.

2. Apply existing search algorithms.

- **States (b-states)**: 2^N possible subsets of the set Σ of N physical states.
- **Initial state**: S .

Solution to a Sensorless Problem

A sequence of actions from search in the space B of *belief states (b-states)*.

♦ **Full observability** in B : the agent always knows its belief state.

1. Transform the physical problem into a belief state problem.

2. Apply existing search algorithms.

- **States (b-states)**: 2^N possible subsets of the set Σ of N physical states.
- **Initial state**: S .
- **Actions**: At the belief state b ,

$$\text{ACTIONS}(b) = \bigcup_{s \in b} \text{ACTIONS}_P(s)$$

Solution to a Sensorless Problem

A sequence of actions from search in the space B of *belief states (b-states)*.

♦ **Full observability** in B : the agent always knows its belief state.

1. Transform the physical problem into a belief state problem.

2. Apply existing search algorithms.

- **States (b-states)**: 2^N possible subsets of the set Σ of N physical states.
- **Initial state**: S .
- **Actions**: At the belief state b ,

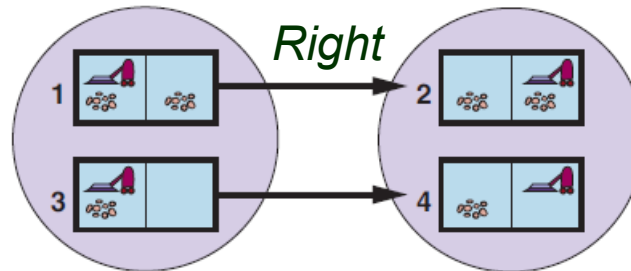
$$\text{ACTIONS}(b) = \bigcup_{s \in b} \text{ACTIONS}_P(s)$$

↑
Action on a physical state

Transition Model

- **Transition model:** The new belief state is, under deterministic actions,

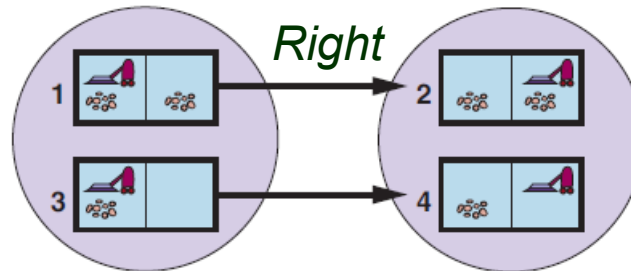
$$b' = \text{RESULT}(b, a) = \{s' \mid s' = \text{RESULT}_p(s, a) \text{ and } s \in b\}$$



Transition Model

- **Transition model:** The new belief state is, under deterministic actions,

$$b' = RESULT(b, a) = \{s' \mid s' = RESULT_P(s, a) \text{ and } s \in b\}$$

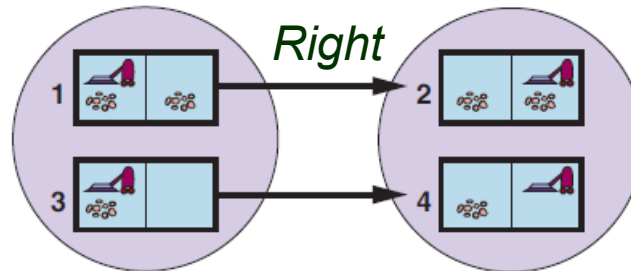


or, under nondeterministic actions,

Transition Model

- **Transition model:** The new belief state is, under deterministic actions,

$$b' = RESULT(b, a) = \{s' \mid s' = RESULT_P(s, a) \text{ and } s \in b\}$$



or, under nondeterministic actions,

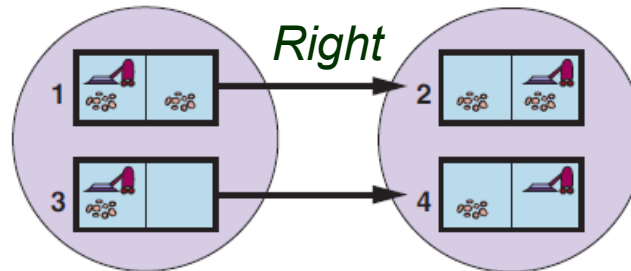
$$b' = RESULT(b, a) = \{s' \mid s' \in RESULT_P(s, a) \text{ and } s \in b\}$$

$$= \bigcup_{s \in b} RESULT_P(s, a)$$

Transition Model

- **Transition model:** The new belief state is, under deterministic actions,

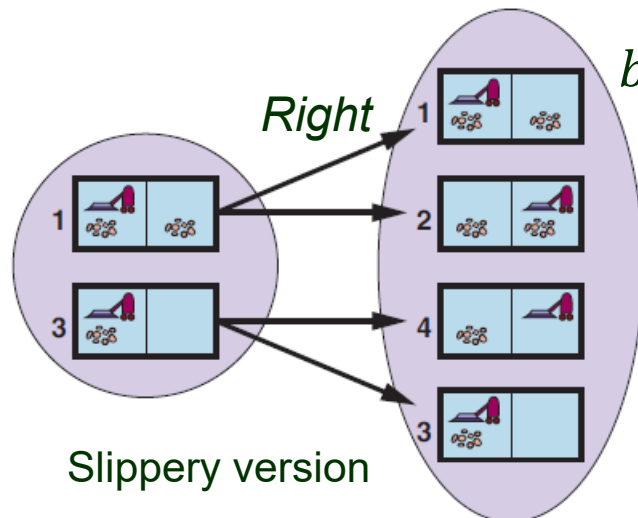
$$b' = RESULT(b, a) = \{s' \mid s' = RESULT_P(s, a) \text{ and } s \in b\}$$



or, under nondeterministic actions,

$$b' = RESULT(b, a) = \{s' \mid s' \in RESULT_P(s, a) \text{ and } s \in b\}$$

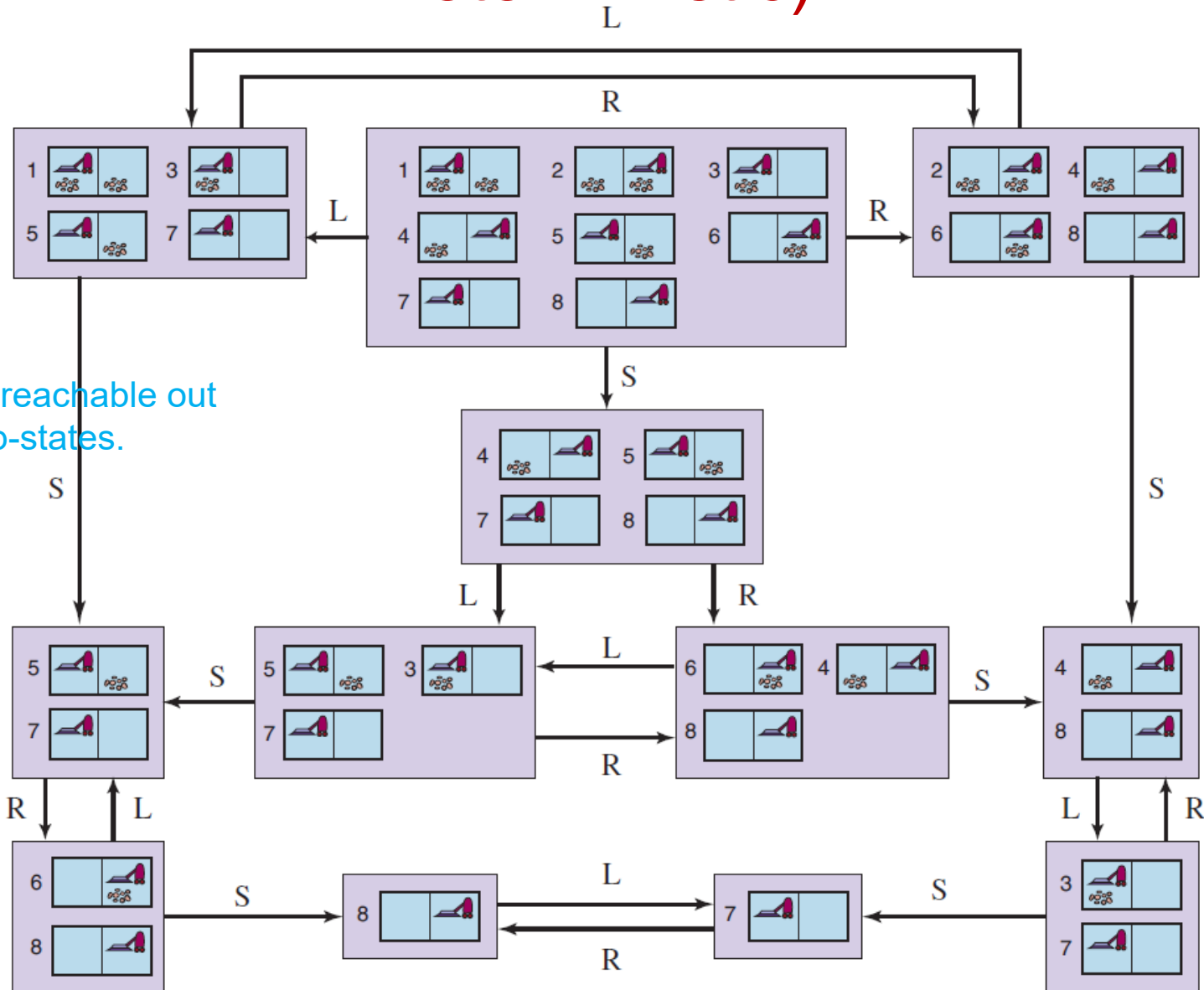
$$= \bigcup_{s \in b} RESULT_P(s, a)$$



Goal & Cost of Action

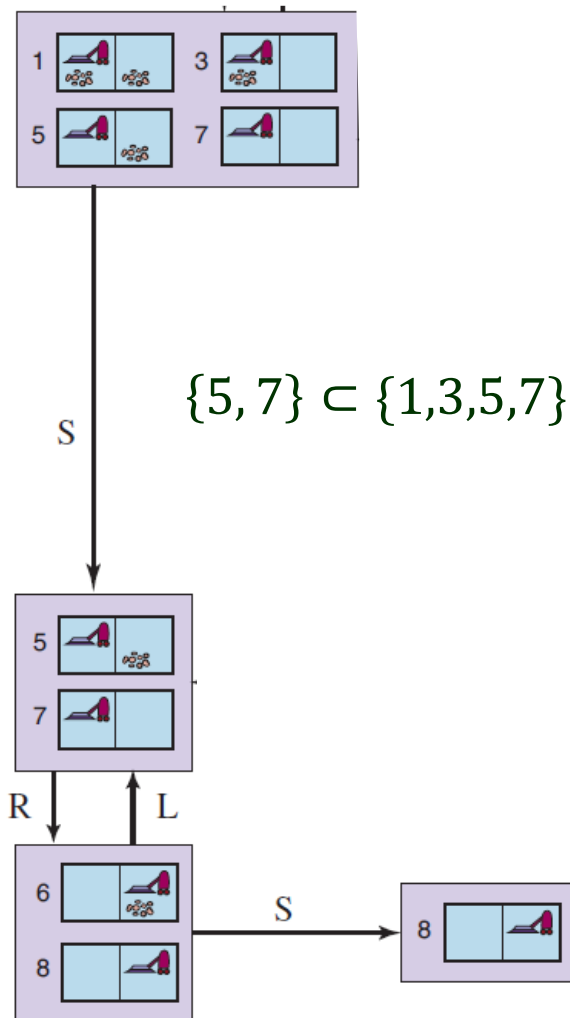
- Goal test: The goal is achieved
 - ♣ *possibly* if one of the states $s \in b$ passes the test;
 - ♣ *necessarily* if every state $s \in b$ passes the test.
- Action cost: Could be one of several values if the same action has different costs in different states.

Reachable Belief States (Sensorless & Deterministic)



Only 12 are reachable out of $256 = 2^8$ b-states.

Subset and Superset States



- ♦ Prune a **superset** b-state, say, $\{1, 3, 5, 7\}$, to concentrate on solving the easier subset b-state, say, $\{5, 7\}$.
- ♦ Prune a **subset** b-state, say, $\{2, 5, 7\}$, if a superset b-state, say, $\{1, 3, 5, 7\}$, of the resulting subset, say, $\{5, 7\}$, has been found to be solvable already.

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)
e.g., Use “nothing” to represent the initial state.

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)
e.g., Use “nothing” to represent the initial state.
- **Incremental belief-state search** to build up solution one physical state at a time.

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)

e.g., Use “nothing” to represent the initial state.

- **Incremental belief-state search** to build up solution one physical state at a time.

♣ Initial state: {1, 2, 3, 4, 5, 6, 7, 8}

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)

e.g., Use “nothing” to represent the initial state.

- **Incremental belief-state search** to build up solution one physical state at a time.

- ♣ Initial state: {1, 2, 3, 4, 5, 6, 7, 8}

- ♣ Find an action sequence that works for state 1.

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)

e.g., Use “nothing” to represent the initial state.

- **Incremental belief-state search** to build up solution one physical state at a time.

- ♣ Initial state: {1, 2, 3, 4, 5, 6, 7, 8}

- ♣ Find an action sequence that works for state 1.

- ♣ Check if it works for state 2.

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)

e.g., Use “nothing” to represent the initial state.

- **Incremental belief-state search** to build up solution one physical state at a time.

- ♣ Initial state: {1, 2, 3, 4, 5, 6, 7, 8}

- ♣ Find an action sequence that works for state 1.

- ♣ Check if it works for state 2.

- If yes, check if it works for state 3, and so on.

Handling Search Space Complexity

N physical states $\Rightarrow 2^N$ belief states!

- Use of compact description, e.g., mathematical logic (Chapter 7)

e.g., Use “nothing” to represent the initial state.

- **Incremental belief-state search** to build up solution one physical state at a time.

- ♣ Initial state: {1, 2, 3, 4, 5, 6, 7, 8}

- ♣ Find an action sequence that works for state 1.

- ♣ Check if it works for state 2.

- If yes, check if it works for state 3, and so on.
 - If no, find a different solution for state 1, and so on.

II. Partially Observable Environments

Many problems (e.g., the 8-puzzle) are unsolvable without sensing.

A little sensing (e.g., only one visible square in the 8-puzzle) can go a long way.

$$\text{PERCEPTS}(s) = \{\text{percept received in the state } s\}$$

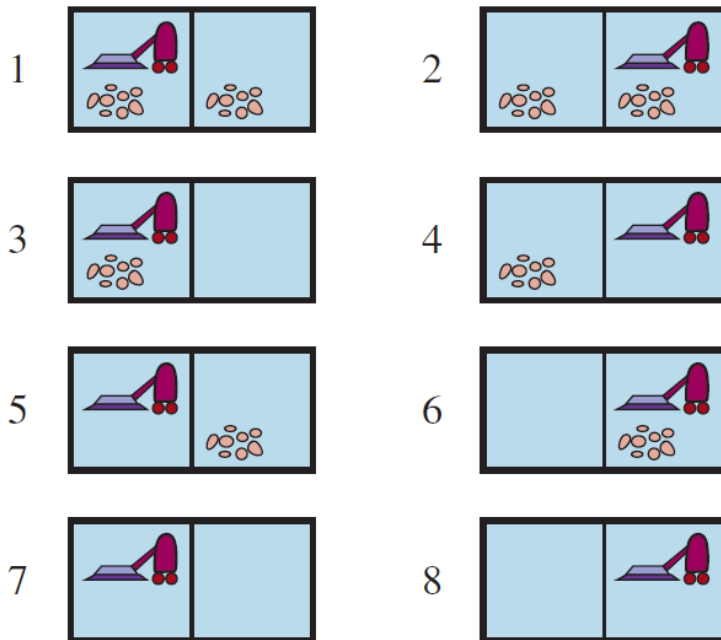
- Several states may yield the same percept.

II. Partially Observable Environments

Many problems (e.g., the 8-puzzle) are unsolvable without sensing.

A little sensing (e.g., only one visible square in the 8-puzzle) can go a long way.

$$\text{PERCEPTS}(s) = \{\text{percept received in the state } s\}$$



- Several states may yield the same percept.

Local-sensing vacuum world: The vacuum cleaner cannot sense the state of the adjacent square.

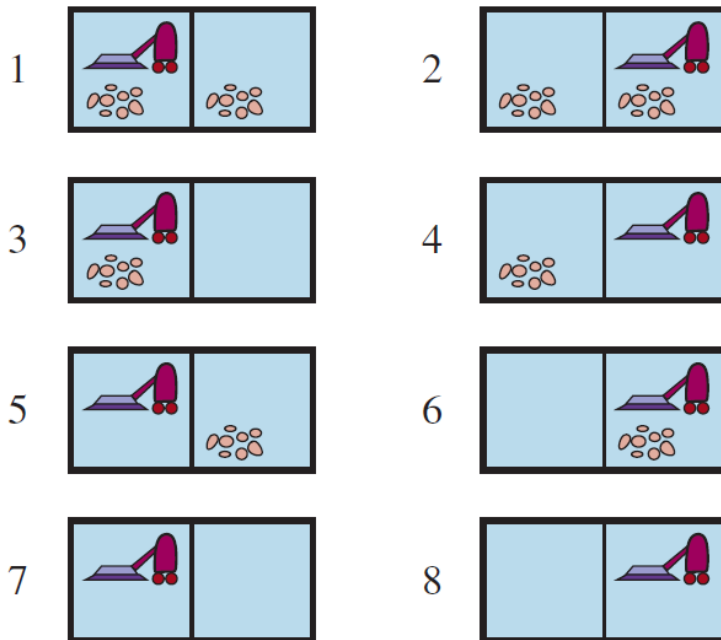
Local-sensing vacuum world

II. Partially Observable Environments

Many problems (e.g., the 8-puzzle) are unsolvable without sensing.

A little sensing (e.g., only one visible square in the 8-puzzle) can go a long way.

$$\text{PERCEPTS}(s) = \{\text{percept received in the state } s\}$$



Local-sensing vacuum world

- Several states may yield the same percept.

Local-sensing vacuum world: The vacuum cleaner cannot sense the state of the adjacent square.

L: in the left square

R: in the right square

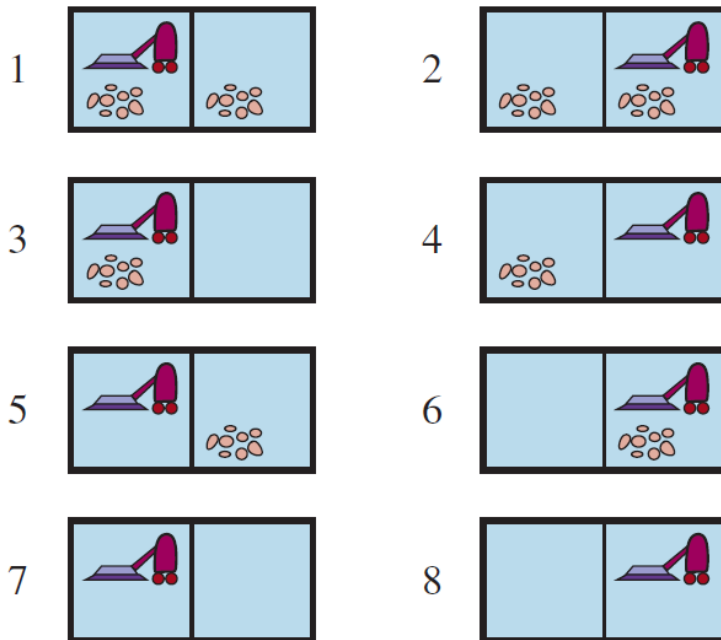
Dirty: current square dirty

II. Partially Observable Environments

Many problems (e.g., the 8-puzzle) are unsolvable without sensing.

A little sensing (e.g., only one visible square in the 8-puzzle) can go a long way.

$$\text{PERCEPTS}(s) = \{\text{percept received in the state } s\}$$



Local-sensing vacuum world

- Several states may yield the same percept.

Local-sensing vacuum world: The vacuum cleaner cannot sense the state of the adjacent square.

L: in the left square

R: in the right square

Dirty: current square dirty

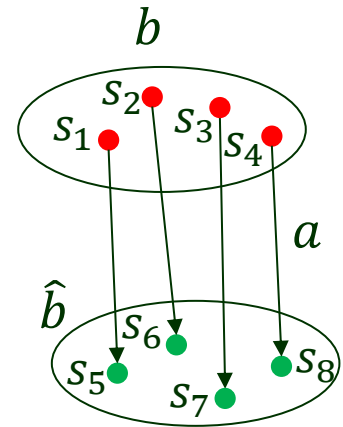
$\{L, \text{Dirty}\} \rightarrow \text{b-state } \{1, 3\}$

Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$



Transition Model Between B-states

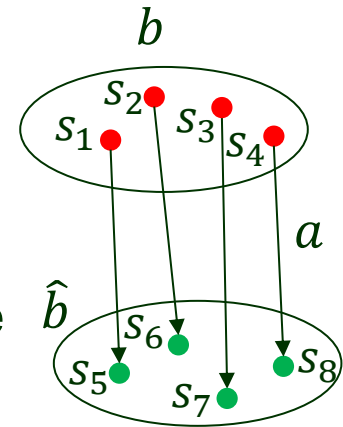
- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$



Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

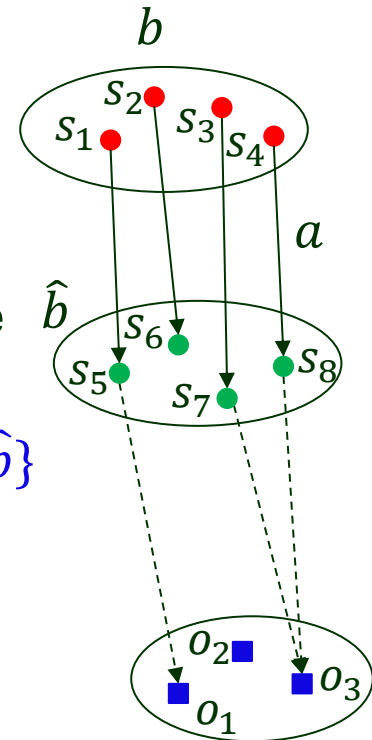
estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $\{o_1, o_3\}$ // state s_6 produces no percept.



Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

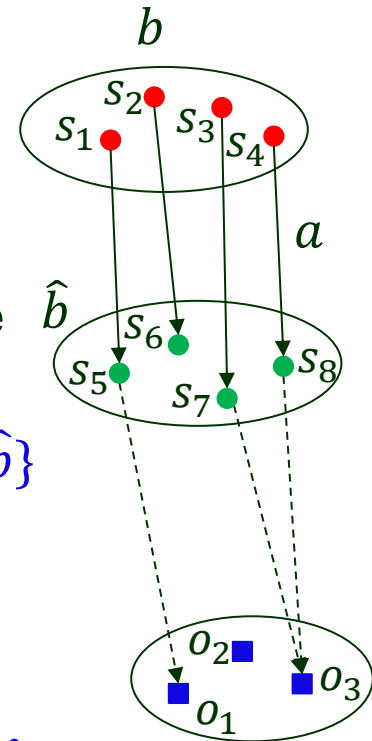
- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $\{o_1, o_3\}$ // state s_6 produces no percept.

- **Update**: computes the set of states in $\hat{b}$ that could have produced a percept o .

$b_o = \text{UPDATE}(\hat{b}, o) = \{s \mid o = \text{POSSIBLE-PERCEPTS}(s) \text{ and } s \in \hat{b}\}$



Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

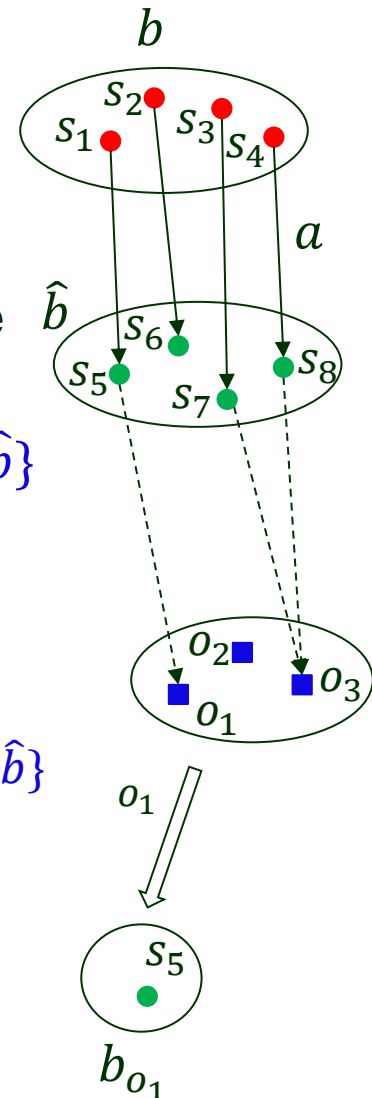
- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $\{o_1, o_3\}$ // state s_6 produces no percept.

- **Update**: computes the set of states in $\hat{b}$ that could have produced a percept o .

$b_o = \text{UPDATE}(\hat{b}, o) = \{s \mid o = \text{POSSIBLE-PERCEPTS}(s) \text{ and } s \in \hat{b}\}$



Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

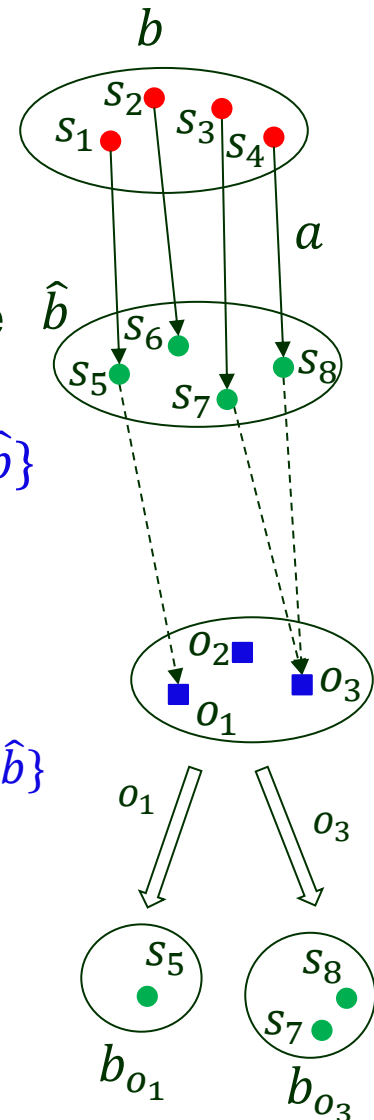
$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $\{o_1, o_3\}$ // state s_6 produces no percept.

- **Update**: computes the set of states in $\hat{b}$ that could have produced a percept o .

$b_o = \text{UPDATE}(\hat{b}, o) = \{s \mid o = \text{POSSIBLE-PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $b_{o_1} = \text{UPDATE}(\hat{b}, o_1) = \{s_5\}$ $b_{o_3} = \text{UPDATE}(\hat{b}, o_3) = \{s_7, s_8\}$



Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow$

$$\hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

$$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$$

e.g., $\{o_1, o_3\}$ // state s_6 produces no percept.

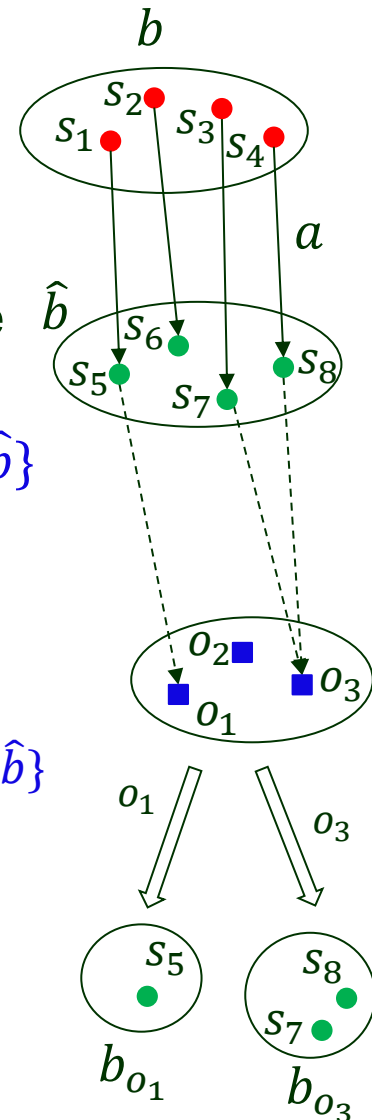
- **Update**: computes the set of states in $\hat{b}$ that could have produced a percept o .

$$b_o = \text{UPDATE}(\hat{b}, o) = \{s \mid o = \text{POSSIBLE-PERCEPTS}(s) \text{ and } s \in \hat{b}\}$$

e.g., $b_{o_1} = \text{UPDATE}(\hat{b}, o_1) = \{s_5\}$ $b_{o_3} = \text{UPDATE}(\hat{b}, o_3) = \{s_7, s_8\}$

- **Result** : resulting belief states

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\hat{b})\}$$



Transition Model Between B-states

- **Prediction**: computes the b-state from an action a .

estimate $\rightarrow \hat{b} = \text{PREDICT}(b, a) \equiv \text{RESULT}(b, a)$

e.g., $b = \{s_1, s_2, s_3, s_4\}$ and $\hat{b} = \{s_5, s_6, s_7, s_8\}$

- **Possible percepts**: computes the set of percepts that could be observed in the predicted b-state.

$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o \mid o = \text{PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $\{o_1, o_3\}$ // state s_6 produces no percept.

- **Update**: computes the set of states in $\hat{b}$ that could have produced a percept o .

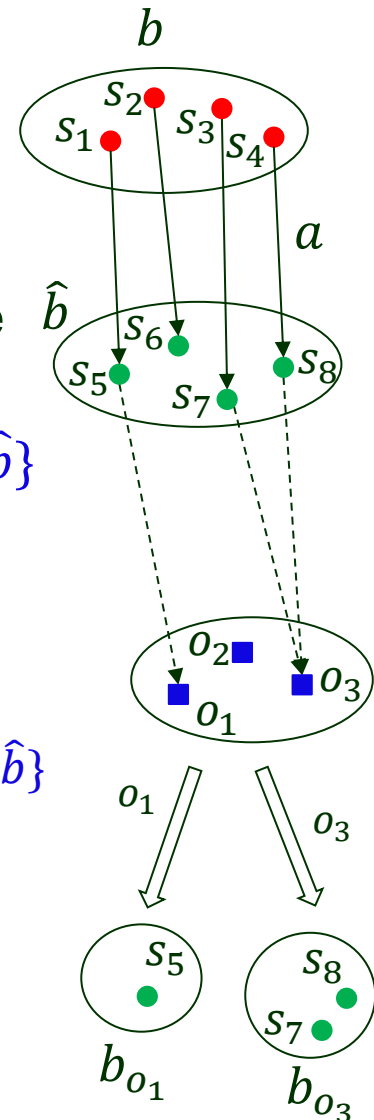
$b_o = \text{UPDATE}(\hat{b}, o) = \{s \mid o = \text{POSSIBLE-PERCEPTS}(s) \text{ and } s \in \hat{b}\}$

e.g., $b_{o_1} = \text{UPDATE}(\hat{b}, o_1) = \{s_5\}$ $b_{o_3} = \text{UPDATE}(\hat{b}, o_3) = \{s_7, s_8\}$

- **Result** : resulting belief states

$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\hat{b})\}$

e.g., $\text{RESULTS}(b, a) = \{b_{o_1}, b_{o_3}\}$



Possible B-states from an Action

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}$$

Includes all the belief states consistent with one possible percept after the action a is applied to the believe state b .

Possible B-states from an Action

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}$$

Includes all the belief states consistent with one possible percept after the action a is applied to the believe state b .

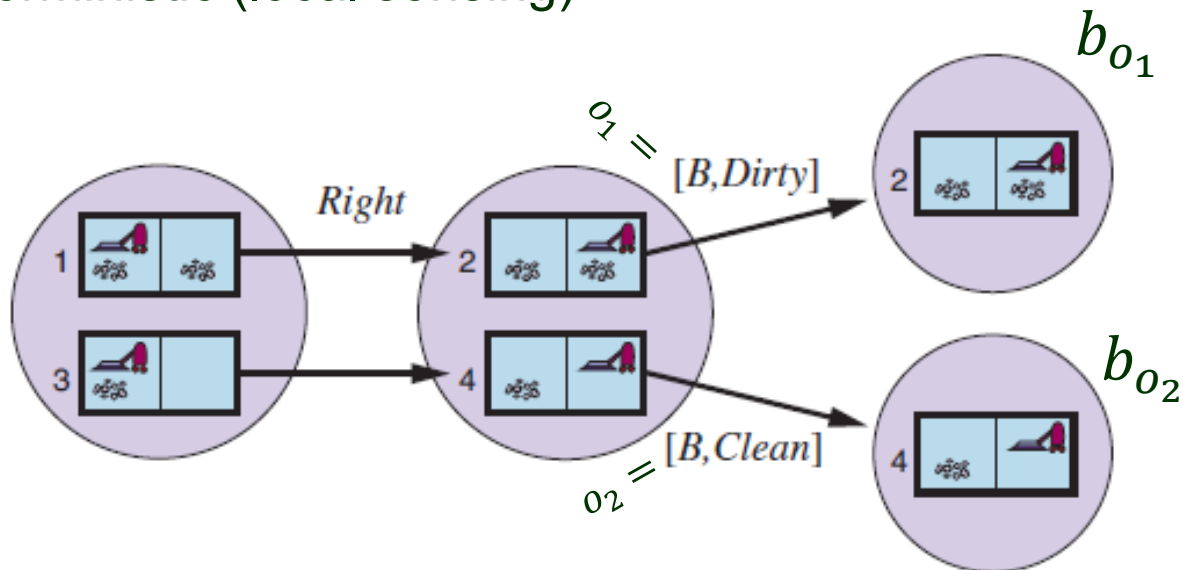
- Deterministic (local sensing)

Possible B-states from an Action

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}$$

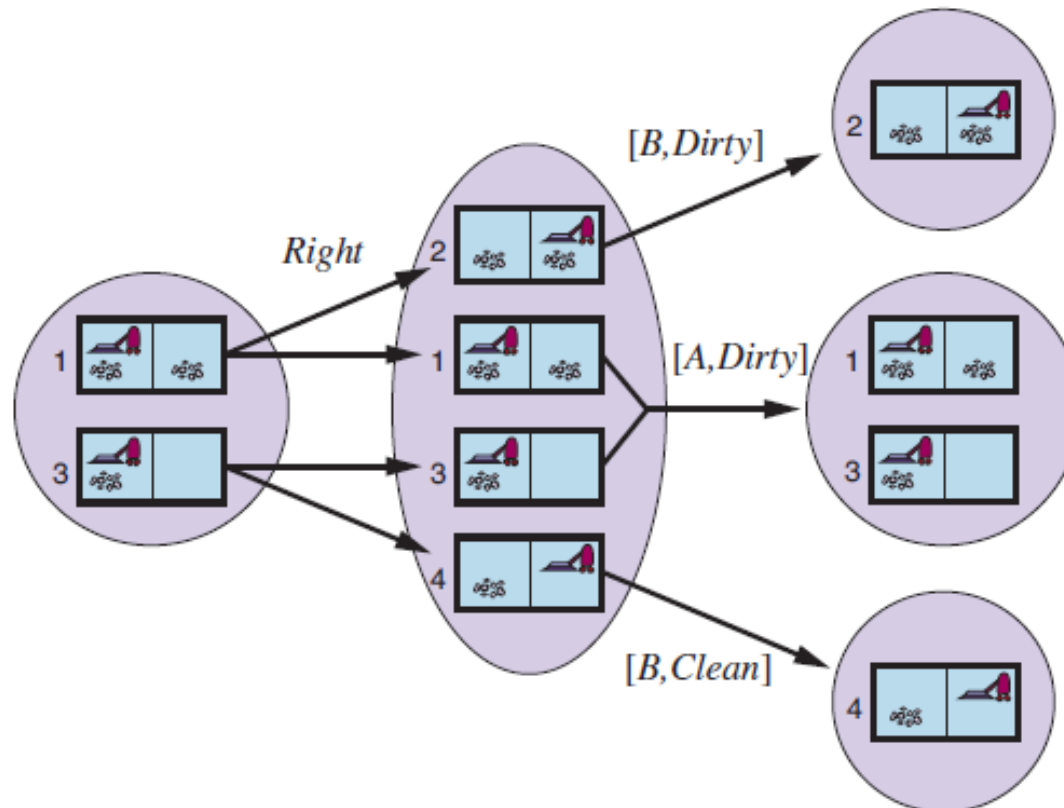
Includes all the belief states consistent with one possible percept after the action a is applied to the believe state b .

- Deterministic (local sensing)



(cont'd)

- Nondeterministic (local sensing + slippery world)

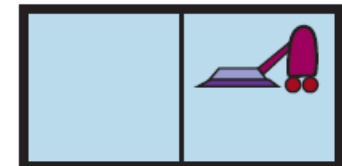
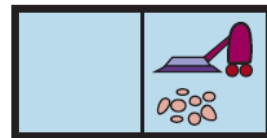


Apply the AND-OR search algorithm.

Initial percept: $[A, \textit{Dirty}]$

```
Suck
Right
if b-state={6}
  then Suck
  else []
```

8



Prediction-Update Cycles

- ♦ Test the condition and execute the appropriate branch.
- ♦ Maintain its belief state as it performs actions and receives percepts.

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Prediction-Update Cycles

- ♦ Test the condition and execute the appropriate branch.
- ♦ Maintain its belief state as it performs actions and receives percepts.

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Consider the local-sensing vacuum world in which

any square may become dirty *any time* unless being actively cleaned.

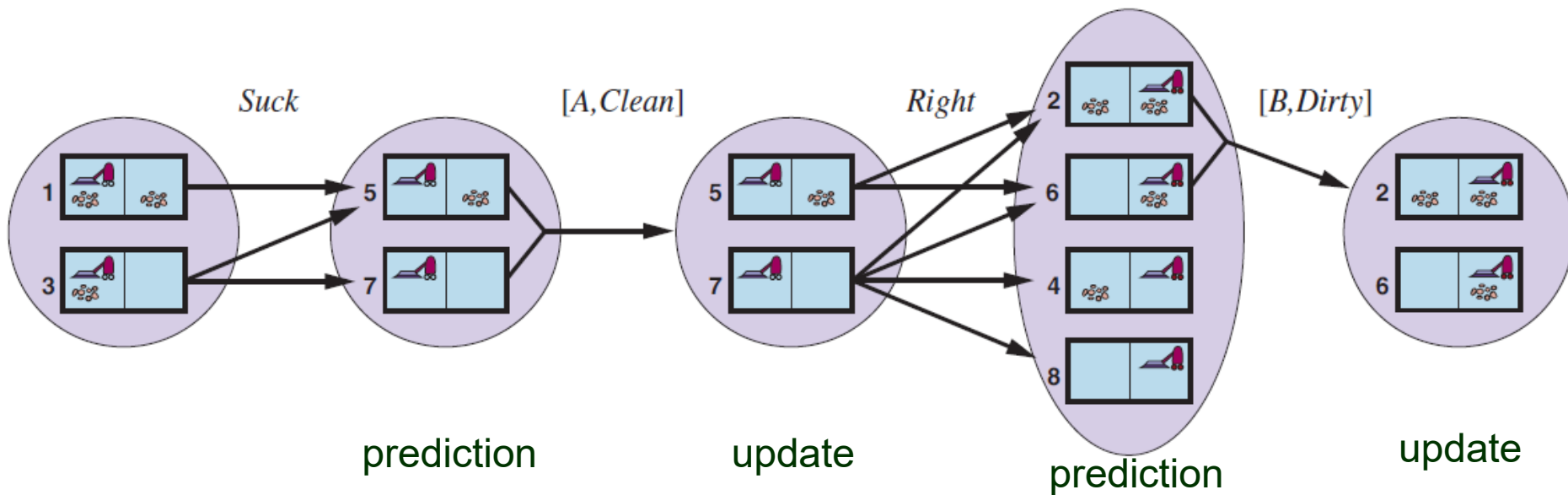
Prediction-Update Cycles

- ♦ Test the condition and execute the appropriate branch.
- ♦ Maintain its belief state as it performs actions and receives percepts.

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Consider the local-sensing vacuum world in which

any square may become dirty *any time* unless being actively cleaned.



Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

↓ *NESW* = 1011 (obstacles in *N*, *S*, *W*)

*b*₀ = UPDATE(*b*, 1011)

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

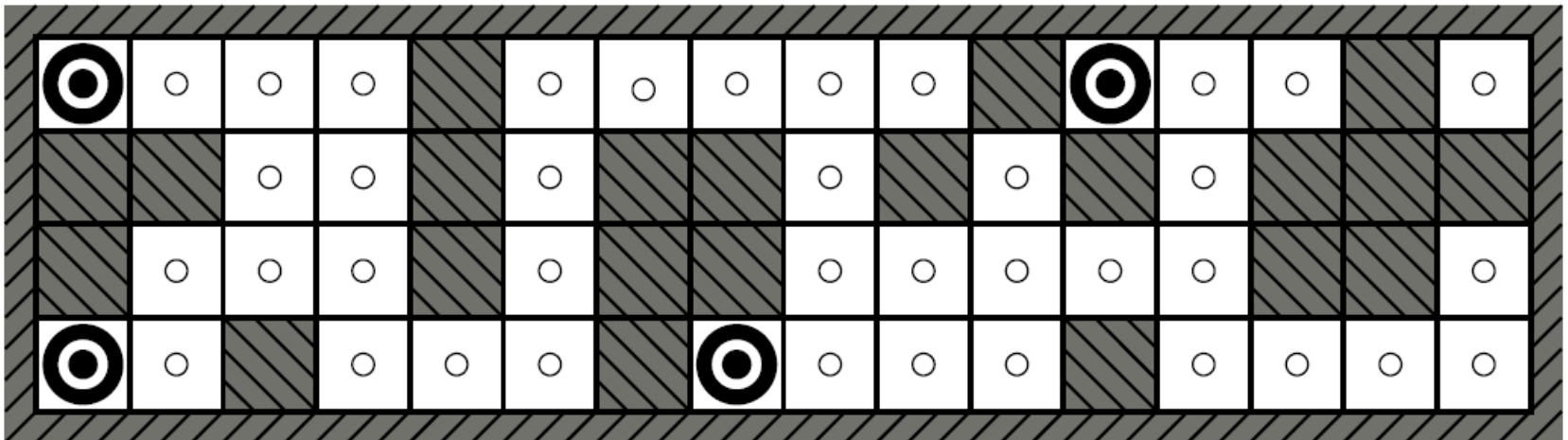
- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b : set of all locations

↓ $NESW = 1011$ (obstacles in N , S , W)

$b_0 = \text{UPDATE}(b, 1011)$



Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

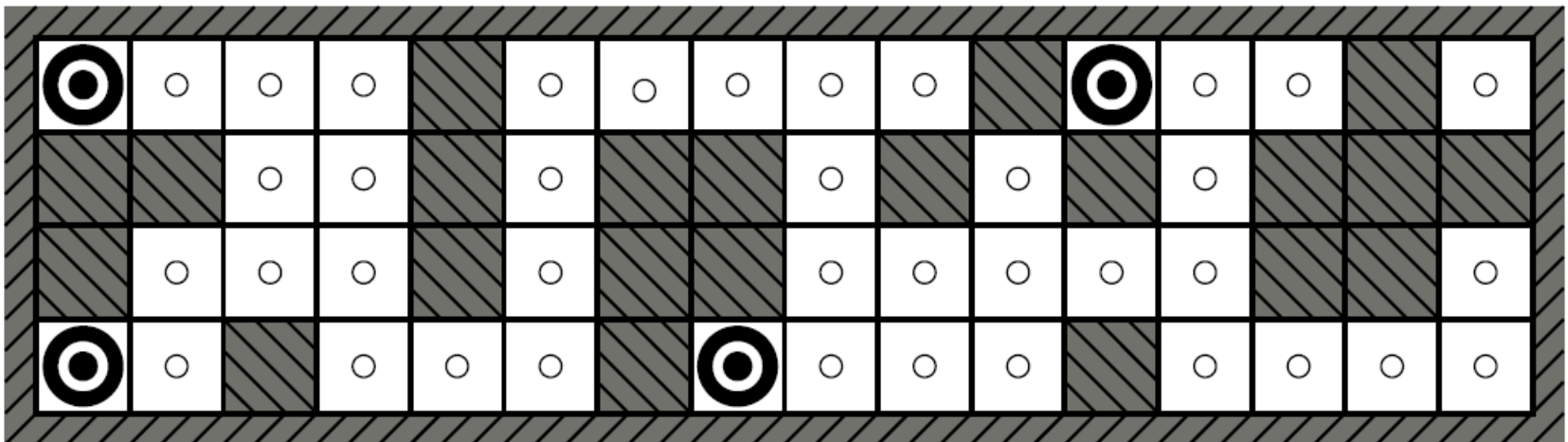
- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

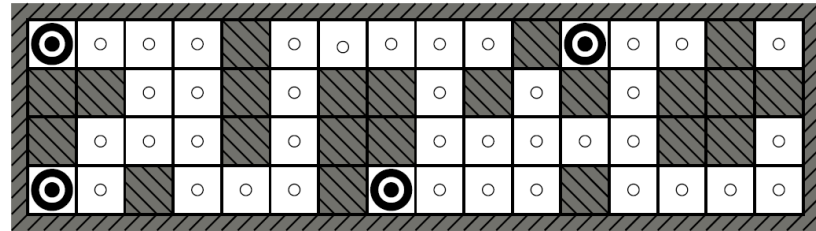
↓ *NESW* = 1011 (obstacles in *N*, *S*, *W*)

*b*₀ = UPDATE(*b*, 1011) // only four possible locations



Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

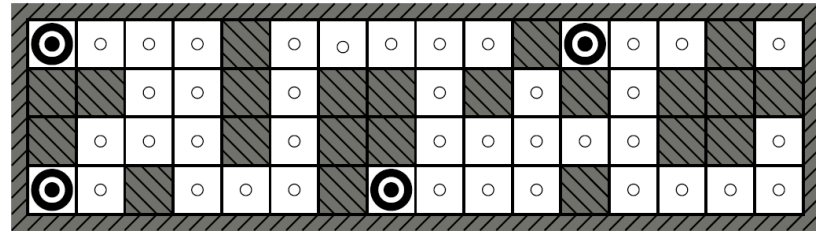


Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

Next, the robot executes a *Right* action.

$$b_a = \text{PREDICT}(b_0, \text{Right})$$



Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

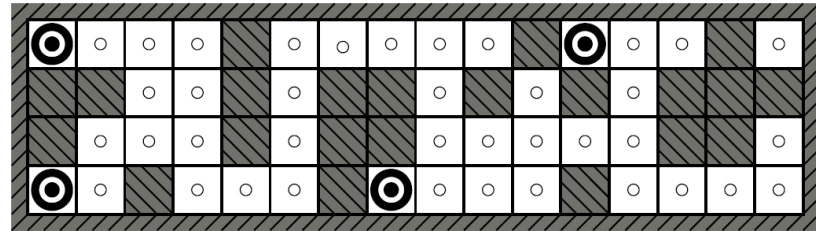
Next, the robot executes a *Right* action.

$$b_a = \text{PREDICT}(b_0, \text{Right})$$



$$NESW = 1010$$

$$b_1 = \text{Update}(b_a, 1010)$$



Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

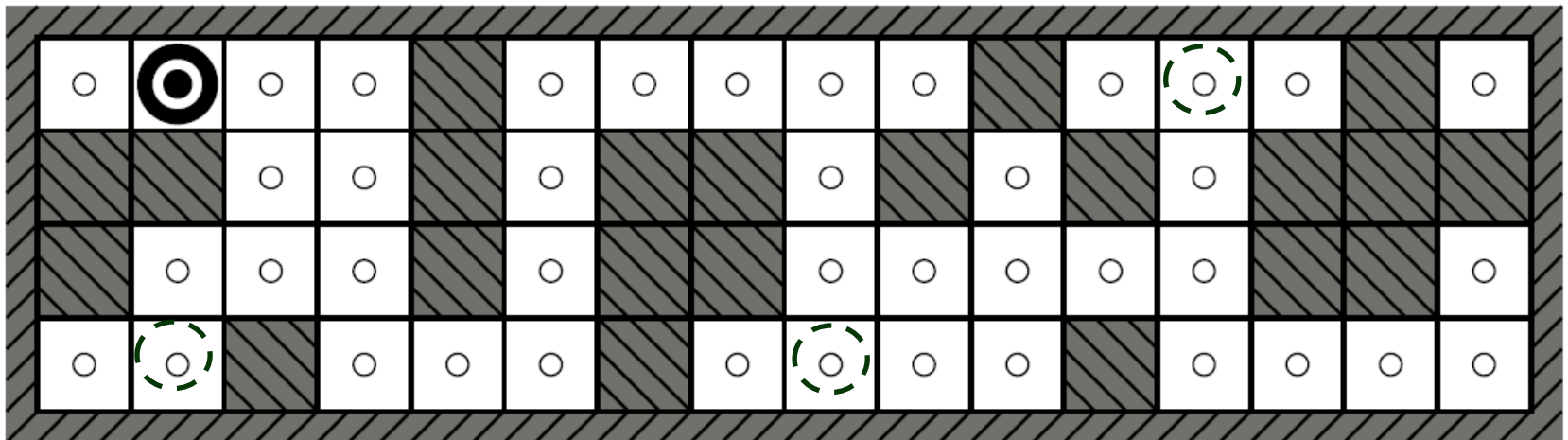
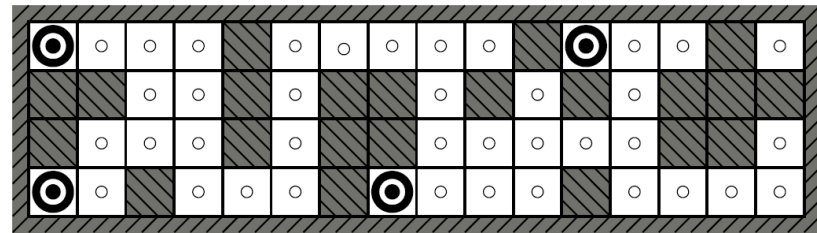
Next, the robot executes a *Right* action.

$$b_a = \text{PREDICT}(b_0, \text{Right})$$



$$\text{NESW} = 1010$$

$$b_1 = \text{Update}(b_a, 1010) \quad // \text{ One possible location!}$$



III. Online Search Agents

Offline search

Compute a complete solution before taking the first action.

III. Online Search Agents

Offline search

Compute a complete solution before taking the first action.

Online search

$\text{RESULT}(s, a)$ is unknown unless being in s and doing a .

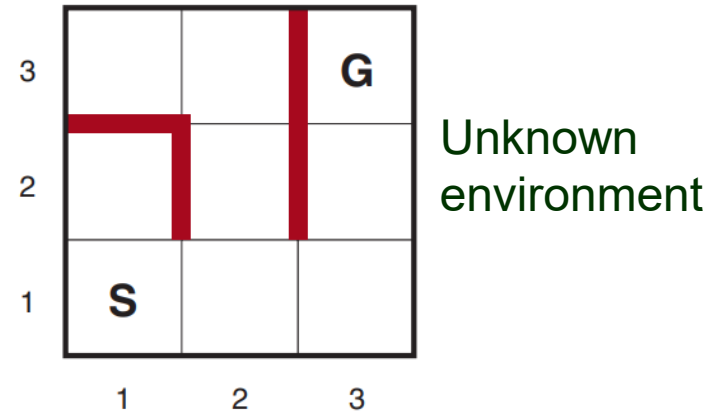
III. Online Search Agents

Offline search

Compute a complete solution before taking the first action.

Online search

$\text{RESULT}(s, a)$ is unknown unless being in s and doing a .



III. Online Search Agents

Offline search

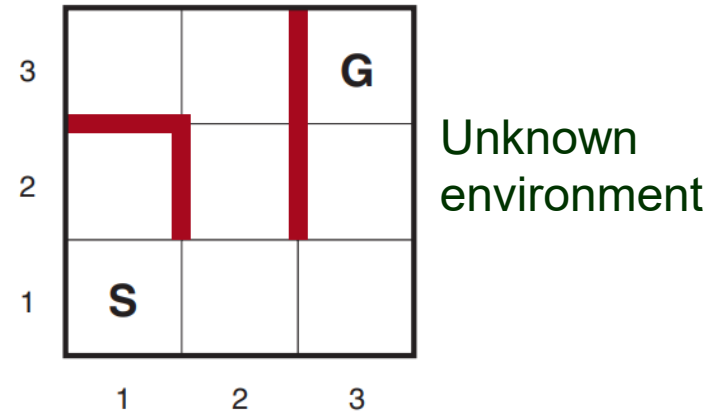
Compute a complete solution before taking the first action.

Online search

$\text{RESULT}(s, a)$ is unknown unless being in s and doing a .

Interleaves computation and action by repeating the steps below:

1. Computes an action.
2. Takes the computed action.
3. Observes the environment.



III. Online Search Agents

Offline search

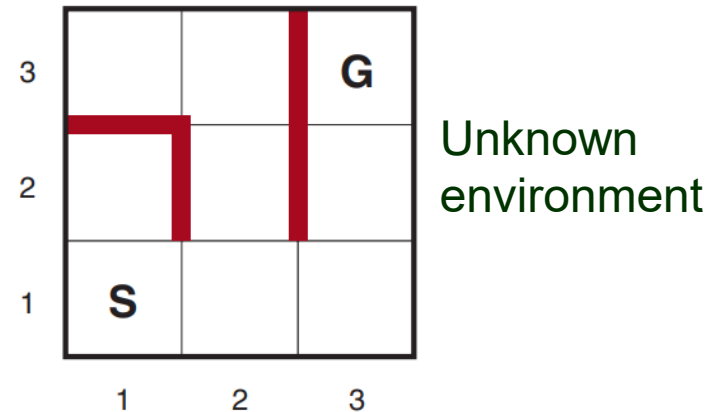
Compute a complete solution before taking the first action.

Online search

$\text{RESULT}(s, a)$ is unknown unless being in s and doing a .

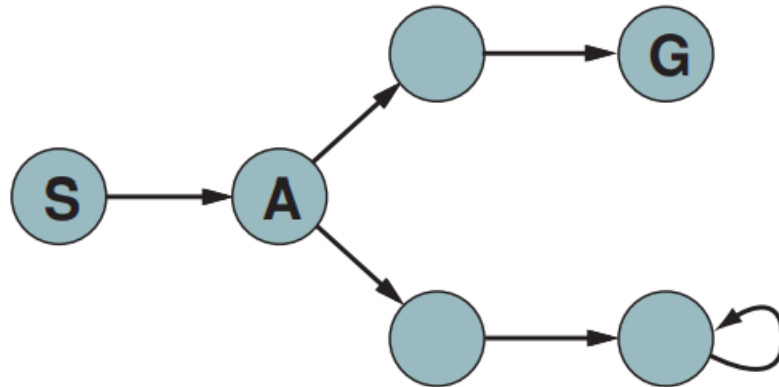
Interleaves computation and action by repeating the steps below:

1. Computes an action.
2. Takes the computed action.
3. Observes the environment.



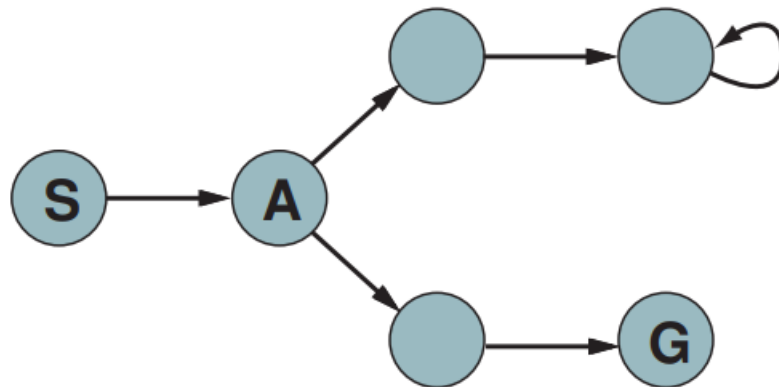
- ◆ Good for search in dynamic or semi-dynamic environments.
- ◆ Allowing the agent to focus on contingencies that actually arise in nondeterministic domains.

Dead Ends



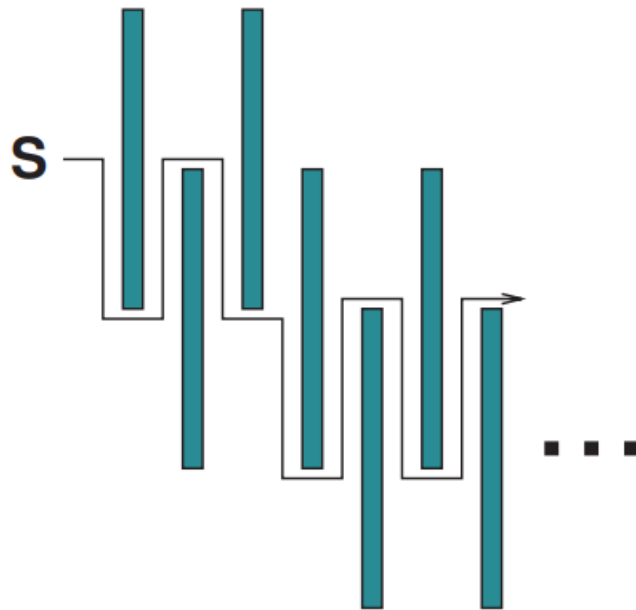
Immediately after visiting S and A :

♠ Cannot tell whether it is in the top state space or the bottom one (because they look identical).



♠ Thus, no action exists to work in both state spaces.

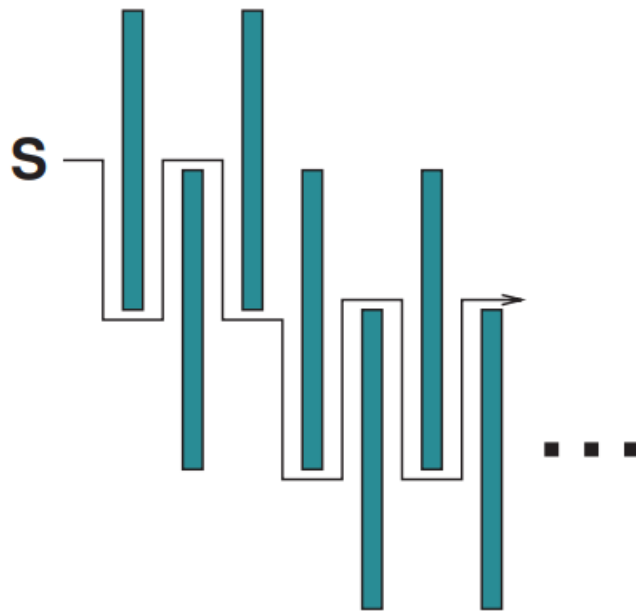
Expensive Search Effort



As if the state space is being constructed by an imaginary adversary.

Whichever actions the agent chooses, dead ends and goals are placed “deliberately” to make the path inefficient.

Expensive Search Effort

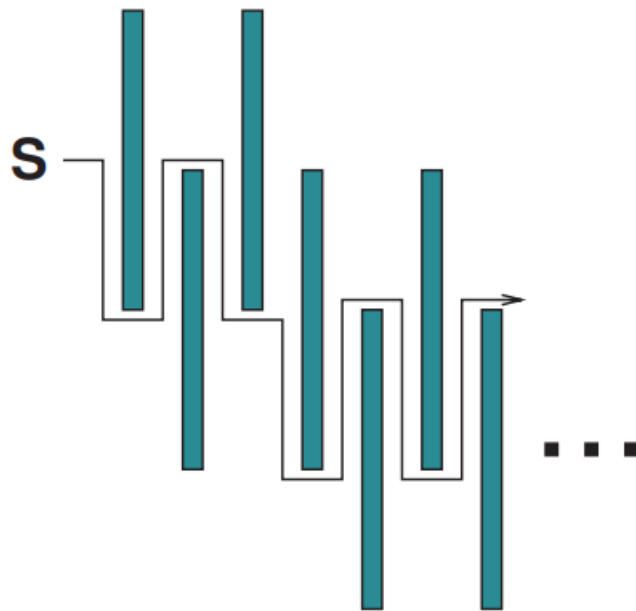


As if the state space is being constructed by an imaginary adversary.

Whichever actions the agent chooses, dead ends and goals are placed “deliberately” to make the path inefficient.

♠ The situation is complicated by that some actions are *irreversible*.

Expensive Search Effort



As if the state space is being constructed by an imaginary adversary.

Whichever actions the agent chooses, dead ends and goals are placed “deliberately” to make the path inefficient.

- ♠ The situation is complicated by that some actions are *irreversible*.
- ♦ Design a search agent that works in *safely explorable* state space.

↑
Every reachable state can reach a goal state..